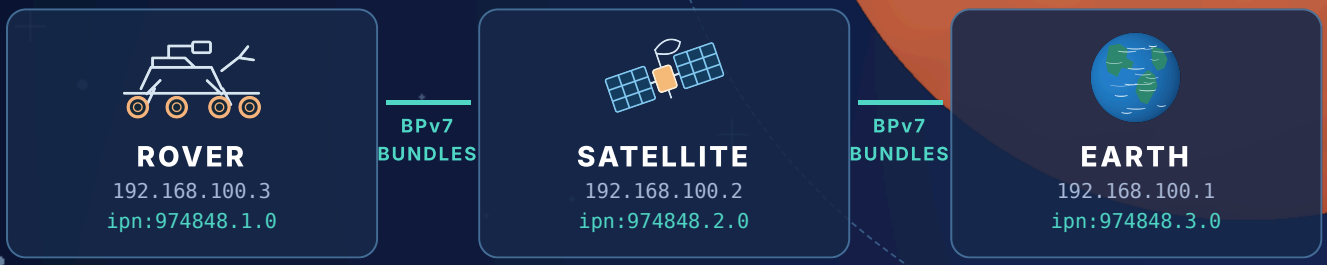
IETF 127 HACKATHON · PARTICIPANT HANDBOOK

CoAP over BPv7

Three-Node Participant Lab Manual

IPN ADDRESSING + MESSAGE AGGREGATION

Download the repository · Configure Rover, Satellite and Earth · Run the live Mission Control GUI · Verify bundles in Wireshark



BPv7 bundles via Satellite · VirtualBox Internal Network: intnet

PROTOTYPE
draft-ietf-core-coap-bp-05 · BPv7 · RFC 9758
three-component IPN · µD3TN/AAP2 · no proxy

HACKATHON BRANCH
`IPN_RFC_9758_Hackathon`
`head reviewed: 3afc82d86e75`

CHAMPIONS
Prof. Anna Calveras · Mahitab Salah Elattar

AFFILIATION
Universitat Politècnica de Catalunya (UPC) · Wireless Networks Group (WNG)

READ BEFORE USING
This guide is for a **fresh participant testbed** running three separate Ubuntu VMs on the same VirtualBox host. Do **not** execute the configuration, build or startup commands over an existing live Hackathon demonstration. The full clean-VM process has not yet been independently end-to-end validated.

MINISTERIO DE CIENCIA, INNOVACIÓN Y UNIVERSIDADES
Cofinanciado por la Unión Europea
AGENCIA ESTATAL DE INVESTIGACIÓN

This work was funded in part by the Spanish MCIU/AEI/10.13039/501100011033/FEDER/UE through project PID2023-146378NB-I00

00 At a glance

The lab runs in three phases. Work through them in order: each phase ends with a check that must pass before you continue.

PHASE 1 · SET UP

- 01 Understand what you are building
- 02 Install prerequisites and download
- 03 Configure the VirtualBox network
- 04 Verify connectivity and clocks
- 05 Build patched µD3TN
- 06 Prepare the Python environment

PHASE 2 · RUN

- 07 Start the three µD3TN nodes
- 08 Configure the forwarding contacts
- 09 Start the Earth server and Rover GUI
- 10 First test: Safety and Science
- 11 Wireshark: see the real bundles

PHASE 3 · EXPERIMENT

- 12 Challenge: ON/OFF connectivity
- 13 Troubleshooting
- 14 Checklist and source links
- A Glossary
- B Results log

CHOOSE YOUR PATH

FULL LIVE LAB

Run all three nodes

One laptop hosts Rover, Satellite and Earth. Follow §01–§12 in order.

Result: real CoAP/BPv7 traffic, logs and captures.

GUI ONLY

Learn Mission Control

Install prerequisites (§02, §06), then open the GUI in **DEMO** mode (§09, optional step).

Result: familiarity with the GUI. Not evidence.

ANALYSIS

Read the bundles

Install the Lua dissector and use the display filters in §11 on a capture shared by the champions.

Result: decoded EIDs, MIDs, Tokens and Aggregates.

TERMINAL MAP — WHAT RUNS WHERE

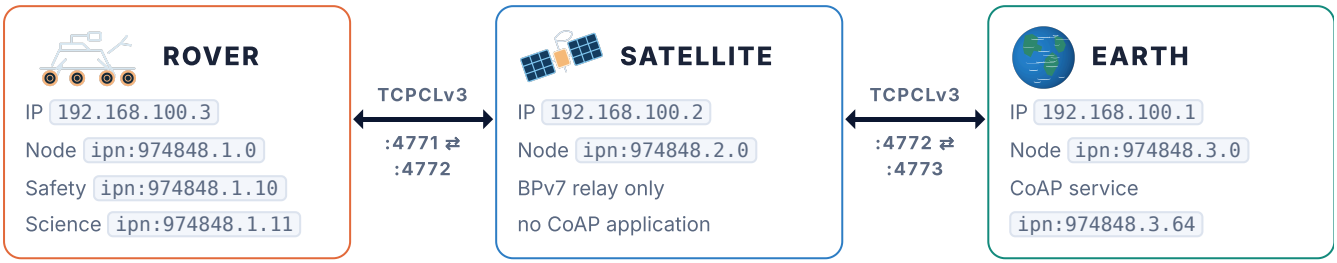
VM	Terminal A · keep open	Terminal B	Terminal C · keep open
SATELLITE start 1st	µD3TN node §07	Contacts §08	—
EARTH start 2nd	µD3TN node §07	Contacts §08	CoAP server §09 · plus a capture terminal §11
ROVER start 3rd	µD3TN node §07	Contacts §08	Mission Control GUI §09

PICK A CONTRIBUTION TRACK

Track	What you do	Start at
Wireshark	Inspect captures; decode EIDs, MIDs, Tokens and Aggregate members	§11
CoAP testing	Exercise Safety GET and Science POST, Single and Aggregate	§10
Contact simulation	Build a safe ON/OFF contact scheduler on an isolated testbed	§08, §12
Measurements	Record what arrives after reconnection across the four runs	§12, App. B
Documentation	Improve this manual: report unclear steps and fixes that worked	§13

01 Understand what you are building

The **Rover** runs two CoAP applications (Safety and Science). The **Satellite** forwards BPv7 bundles. The **Earth** node runs the CoAP server. Messages traverse TCPCLv3 links between µD3TN nodes; applications interface with µD3TN through AAP2. There is no CoAP/UDP proxy.



All three VMs on one VirtualBox host · Internal Network: **intnet** · 192.168.100.0/24

VM / role	Internal IP	Node EID	TCPCLv3	AAP2 socket suffix
ROVER	192.168.100.3/24	ipn:974848.1.0	4771	rover.aap2.sock
SATELLITE	192.168.100.2/24	ipn:974848.2.0	4772	sat.aap2.sock
EARTH	192.168.100.1/24	ipn:974848.3.0	4773	earth.aap2.sock

Node EID ≠ application EID
Application EIDs: Safety = `ipn:974848.1.10`; Science = `ipn:974848.1.11`; Earth CoAP service = `ipn:974848.3.64`. The Earth **application** EID is different from the Earth **node** EID (`ipn:974848.3.0`). Each EID reads *allocator.node.service* (RFC 9758).

You will need

- A host PC capable of running **three Ubuntu VMs simultaneously** in VirtualBox; all three VMs must use the **same host** for the intnet internal network.
- Ubuntu with **Python 3.10+** and Tkinter. Ubuntu 22.04 LTS is a practical starting point, but this lab has not been validated there end to end; the original working lab used Ubuntu 20.04 with a separately installed Python 3.10.
- Internet connectivity to download source and Python packages, and an Ubuntu terminal on each VM.
- Basic familiarity with `cd`, `bash` and opening additional terminals.

Keep LIVE and DEMO modes separate
LIVE mode sends real CoAP/BPv7 traffic and requires the entire network. **DEMO** mode displays simulated activity; it is useful for learning the GUI, but its measurements are not experimental evidence.

02 Install prerequisites and download the code

RUN ON **ALL THREE VMs**

Repeat this step on Rover, Satellite and Earth. The branch must be `IPN_RFC_9758_Hackathon`; the default `main` may contain older allocator-0 code and GUI scripts.

ALL THREE VMs A · Install the required Ubuntu packages

```
sudo apt update
sudo apt install -y git build-essential libsqlite3-dev \
  python3.10 python3.10-venv python3-tk python3-pip \
  iproute2 iputils-ping
```

ALL THREE VMs Confirm Python and Tkinter

```
python3.10 --version
python3.10 -c "import tkinter; print('Tkinter OK')"
```

Ubuntu 22.04 typically includes Python 3.10 packages. If your Ubuntu release does not provide `python3.10`, use a maintained Python 3.10+ installation and set `PYTHON_BIN` when running the Python setup script. Do not replace the system Python.

ALL THREE VMs B · Download the exact GitHub branch

```
cd ~
git clone --branch IPN_RFC_9758_Hackathon --single-branch \
  https://github.com/ENTEL-WNG/CoAPoverBP-IPN-Draft05-NoProxy.git
cd CoAPoverBP-IPN-Draft05-NoProxy
```

ALL THREE VMs C · Check the downloaded files

```
git branch --show-current
ls scripts/prepare_participant_ud3tn.sh
ls scripts/setup_participant_python.sh
ls gui-v2/IETF127_Experiment_C_Package/run_gui_congestion_results.sh
```

Pass when

The branch command prints `IPN_RFC_9758_Hackathon` and each `ls` command shows its filename. A GitHub permission or authentication error means the participant may need repository access.

Repository link

Open the exact Hackathon branch →

`https://github.com/ENTEL-WNG/CoAPoverBP-IPN-Draft05-NoProxy/tree/IPN_RFC_9758_Hackathon`

03 Configure the VirtualBox network

RUN ON **ALL THREE VMs**

Each VM needs two network adapters: one for downloading software, and one private link shared by the three lab nodes.

A VirtualBox settings (before starting each new VM)

- Open VirtualBox → select the VM → **Settings → Network**.
- **Adapter 1:** Enabled; Attached to **NAT** (Internet/download access).
- **Adapter 2:** Enabled; Attached to **Internal Network**; Name: `intnet`; Cable Connected.
- Use the same internal network name for Rover, Satellite and Earth. Start the three VMs.

B Discover the interface

ALL THREE VMs

```
ip -br link
ip -br -4 addr
```

The participant helper assumes the internal interface is `enp0s8`. If yours differs, set `IETF127_NET_IFACE` to the actual internal adapter name before calling the helper.

C Configure each VM — only if fresh and idle

From the downloaded repository root, run the two commands for that VM. The check is read-only; the configure action requires an explicit fresh-VM safeguard.

VM	1 · Read-only check	2 · Configure IP if missing
ROVER	`bash scripts/participant_vm_network.sh --check rover`	`IETF127_FRESH_VM=YES bash scripts/participant_vm_network.sh --configure rover`
SATELLITE	`bash scripts/participant_vm_network.sh --check satellite`	`IETF127_FRESH_VM=YES bash scripts/participant_vm_network.sh --configure satellite`
EARTH	`bash scripts/participant_vm_network.sh --check earth`	`IETF127_FRESH_VM=YES bash scripts/participant_vm_network.sh --configure earth`

For a custom internal interface, first run `export IETF127_NET_IFACE=YOUR_INTERFACE`. The helper refuses changes when an active Hackathon µD3TN socket or TCPCL listener is detected, and it refuses to overwrite an existing IPv4 address.

> **IP addresses are temporary**
> The helper adds the role-specific IPv4 address for the current boot only. If the participant VMs restart, rerun the read-only checks; addresses may need to be reconfigured. **Do not run the configure action on the presenter's working VMs.**

04 Verify connectivity and clocks

RUN ON **ALL THREE VMs**

Run these checks after all three internal addresses are configured. They are read-only.

A Test reachability from each VM

ROVER

```
ping -c 3 192.168.100.2
ping -c 3 192.168.100.1
```

SATELLITE

```
ping -c 3 192.168.100.3
ping -c 3 192.168.100.1
```

EARTH

```
ping -c 3 192.168.100.2
ping -c 3 192.168.100.3
```

Pass when
The other VMs respond to ping. If not, correct VirtualBox Adapter 2 and the interface IP assignments before attempting to start µD3TN.

B Check each clock

ALL THREE VMs

```
date -u
timedatectl status
```

The three VM clocks should be closely synchronized. BPv7 time stamps and bundle lifetimes can be affected by significant clock differences. Use the OS time-sync facility appropriate for the participant environment; do not manually change clocks mid-experiment.

C Confirm the source copy belongs to this VM

ALL THREE VMs

```
pwd
git branch --show-current
```

Pass when
Working folder is `~/CoAPoverBP-IPN-Draft05-NoProxy` and the branch is `IPN_RFC_9758_Hackathon`. Keep this same directory for all subsequent commands.

Important VirtualBox limitation
An Internal Network called intnet connects VMs on the **same VirtualBox host**; it does not connect separate participants' laptops over the Internet. For the standard exercise, one participant runs all three VMs on one host.

05 Build patched µD3TN on all three VMs

RUN ON **ALL THREE VMs**

The Hackathon repository includes a nine-file IPN/AAP2 patch, but it does not include a ready-to-run patched µD3TN binary. Each new VM needs a separate copy of the matching original project and a successful build.

ALL THREE VMs A · Read-only prerequisite check

```
cd ~/CoAPoverBP-IPN-Draft05-NoProxy
bash scripts/prepare_participant_ud3tn.sh --check
```

ALL THREE VMs B · Clone the pinned original and build the patched binary

```
bash scripts/prepare_participant_ud3tn.sh --install
```

This script clones **ENTEL-WNG/CoAPoverBP-no-proxy-version** into a new `~/ietf127-participant/` workspace, checks the nine-file patch, applies it and runs the POSIX build. It is designed to stop on patch/build failures instead of silently continuing.

ALL THREE VMs C · Verify the binary exists

```
test -x "$HOME/ietf127-participant/CoAPoverBP-no-proxy-version/ud3tn/"\
"build/posix/ud3tn" && echo "Patched µD3TN binary: OK"
```

Pass when
The command prints **Patched µD3TN binary: OK**. The expected executable is `~/ietf127-participant/CoAPoverBP-no-proxy-version/ud3tn/build/posix/ud3tn`.

Do not use make run-posix
That would start an additional daemon outside the controlled three-VM launch order. Nodes are started only in §07.

Build verification scope
The original repository's nine pre-patch file hashes matched the nine files referenced by the supplied patch. That is stronger than an unverified source guess, but the complete fresh C build has not been independently tested. If a build fails, save the error output and do not force-apply the patch.

06 Prepare the Python environment

RUN ON **ALL THREE VMs**

Run this on each new VM, from the Hackathon repository root. The helper script exports the correct `UD3TN_ROOT` path and selects the repository's own `.venv` Python environment.

ALL THREE VMs A · Load the participant environment

```
cd ~/CoAPoverBP-IPN-Draft05-NoProxy
source scripts/participant_env.sh
```

At this point, a warning that `.venv` is missing is normal. The next steps create it.

B Required Protobuf compatibility correction (current branch)

The inspected `setup_participant_python.sh` installs `protobuf==5.27.2`, but the pinned upstream AAP2 generated file declares 5.28.3. On the participant's new clone only, update the version before installing:

ALL THREE VMs

```
sed -i 's/protobuf==5[.]27[.]2/protobuf==5.28.3/' \
  scripts/setup_participant_python.sh
grep protobuf scripts/setup_participant_python.sh
```

Pass when
The output contains `protobuf==5.28.3`. Once the GitHub script is corrected upstream, the `sed` command will be unnecessary.

ALL THREE VMs C · Install and check the Python-side live dependencies

```
bash scripts/setup_participant_python.sh --live
```

The script creates `.venv`, installs Python packages and verifies that Tkinter, the vendored modified aiocoap and the AAP2 bindings can be imported. It does not connect any VMs or send bundles.

ALL THREE VMs D · Confirm the imports from the same terminal

```
.venv/bin/python -c "import aiocoap, aioconsole; \
from ud3tn_utils.aap2.generated import aap2_pb2; \
print('Live Python imports: OK')"
```

Pass when
The command prints **Live Python imports: OK**.

Why the aiocoap fork matters
The experiment uses a modified aiocoap source from `vendor/aiocoap/src`, including a 24-bit MID and an experimental Payload-length option. Do not replace it with an unrelated stock aiocoap installation.

07 Start the three µD3TN nodes

RUN ON SATELLITE EARTH ROVER

Do these steps only after the VM network, µD3TN build and Python setup have passed their checks. Start the nodes **in this order**. Each node runs in its own long-lived terminal.

1 Satellite first

SATELLITE TERMINAL A

```
cd ~/CoAPoverBP-IPN-Draft05-NoProxy
source scripts/participant_env.sh
bash scripts/start_ud3tn.sh satellite
```

Expected: node `ipn:974848.2.0`, TCPCLv3 listener 4772, AAP2 socket `/tmp/ud3tn-974848-sat.aap2.sock`. Leave this terminal open.

2 Then Earth

EARTH TERMINAL A

```
cd ~/CoAPoverBP-IPN-Draft05-NoProxy
source scripts/participant_env.sh
bash scripts/start_ud3tn.sh earth
```

Expected: node `ipn:974848.3.0`, TCPCLv3 listener 4773, AAP2 socket `/tmp/ud3tn-974848-earth.aap2.sock`. Leave this terminal open.

3 Then Rover

ROVER TERMINAL A

```
cd ~/CoAPoverBP-IPN-Draft05-NoProxy
source scripts/participant_env.sh
bash scripts/start_ud3tn.sh rover
```

Expected: node `ipn:974848.1.0`, TCPCLv3 listener 4771, AAP2 socket `/tmp/ud3tn-974848-rover.aap2.sock`. Leave this terminal open.

Read-only status check (another terminal)

ALL THREE VMs

```
ss -ltn | grep -E ":(4771|4772|4773)[[:space:]]"
ls -l /tmp/ud3tn-974848-*.aap2.sock
```

Pass when
Each VM shows its own TCPCLv3 listener and its AAP2 socket file.

Do not duplicate services
If a node is already running, do not run its startup command again. A second copy could collide with the TCPCL listener and AAP2 socket. No script in this guide should be run over the presenter's active demonstration.

08 Configure the forwarding contacts

RUN ON SATELLITE EARTH ROVER

A **contact** is a scheduled opportunity for µD3TN to forward bundles toward another node. The repository scripts configure a two-way route through the Satellite. Run each command in a new **Terminal B** while each µD3TN daemon keeps running in Terminal A.

1 Satellite

SATELLITE TERMINAL B

```
cd ~/CoAPoverBP-IPN-Draft05-NoProxy
source scripts/participant_env.sh
bash scripts/satellite_contacts.sh
```

Establishes Satellite → Earth via `192.168.100.1:4773` and Satellite → Rover via `192.168.100.3:4771`.

2 Earth

EARTH TERMINAL B

```
cd ~/CoAPoverBP-IPN-Draft05-NoProxy
source scripts/participant_env.sh
bash scripts/earth_contacts.sh
```

Establishes Earth → Satellite, including reachability toward Rover.

3 Rover

ROVER TERMINAL B

```
cd ~/CoAPoverBP-IPN-Draft05-NoProxy
source scripts/participant_env.sh
bash scripts/rover_contacts.sh
```

Establishes Rover → Satellite, including reachability toward Earth.

Resulting bundle paths

REQUEST: Safety / Science app → Rover µD3TN → Satellite → Earth CoAP server

RESPONSE: Earth CoAP server → Earth µD3TN → Satellite → Rover application

Contacts expire after one hour

The committed contact scripts use `--schedule 1 3600 100000`: a contact starting near now, with a 3,600-second duration and a configured volume parameter. Contacts can expire. These default scripts do **not** create an ON/OFF experiment automatically.

09 Start the Earth server and Rover GUI

RUN ON EARTH ROVER

With all nodes and contacts in place, start the two CoAP endpoints: the server on Earth, then the Mission Control GUI on the Rover.

EARTH TERMINAL C · KEEP RUNNING A · Earth CoAP server

```
cd ~/CoAPoverBP-IPN-Draft05-NoProxy
source scripts/participant_env.sh
bash scripts/earth_server.sh
```

This starts `applications/earth/NodeB_Mission_Draft05_IPN_Allocator974848.py`, registering the Earth CoAP service `ipn:974848.3.64`. Leave the terminal open to observe incoming requests and outgoing responses.

ROVER TERMINAL C B · The real Mission Control GUI

```
cd ~/CoAPoverBP-IPN-Draft05-NoProxy
source scripts/participant_env.sh
cd gui-v2/IETF127_Experiment_C_Package
bash ./run_gui_congestion_results.sh
```

This is the same launcher used in the presenter's working setup. It defaults to **LIVE** mode and loads `mission_control_v2_live_metrics.py` with the real Rover application engine and AAP2 socket. The GUI uses the Safety application at `ipn:974848.1.10`, the Science application at `ipn:974848.1.11` and the Earth CoAP service `ipn:974848.3.64`. Its initial **NSTART** setting is 1.

Optional: GUI-only simulation

If the three-node network is not ready, you can still open the same GUI with simulated values. This is a learning aid, not a replacement for live testing.

ROVER DEMO mode

```
cd ~/CoAPoverBP-IPN-Draft05-NoProxy/gui-v2/IETF127_Experiment_C_Package
bash ./run_gui_congestion_results.sh --demo
```

Check the mode
A missing AAP2 socket in LIVE mode is an installation/startup problem; the launcher does not silently switch into DEMO mode. Do not interpret demonstration numbers as measured BPv7 results.

10 First test: real Safety and Science requests

RUN ON ROVER EARTH

Send one request of each kind as a Single, confirm the round trip in the logs, and only then try an Aggregate.

A Safety — a Single NON GET

- In the LIVE Mission Control GUI, select **Safety** and send a GET request for `/mission/safety-guidance`.
- Watch the **Rover GUI**: request submitted, AAP2 bundle sent, then the response (when the contacts deliver it).
- Watch the **Earth server terminal**: request source should be `ipn:974848.1.10`; Earth service EID `ipn:974848.3.64`.
- The application should receive a **NON Content** response with a matching CoAP Token. Check real logs instead of assuming success from a button animation.

B Science — a Single NON POST

- Select **Science** and send an observation to `/science/observations`.
- Earth should decode the JSON observation, log it, and send a separate **NON 2.01 Created** response.
- Verify the response is matched at Rover using the Token, and observe the corresponding BPv7 traffic in Wireshark on Earth (§11).

C Experimental aggregation — two Science NON POSTs

- Only after Singles work, use the GUI's controlled experimental aggregation / **NSTART=2** setting, subject to the project's congestion-control caution.
- Submit two Science requests from the **same application EID** to the same Earth CoAP service within the engine's aggregation window.
- Verify **one BPv7 request bundle with two CoAP messages**, each with a distinct MID/Token and a Payload-length option; Earth should process both and send separate responses.

Observed example from the prepared laboratory: a 212-byte Science Aggregate containing two 50-byte payload members. That was one recorded run, not a fixed size for every installation.

What counts as evidence?

Claim	Evidence required
Submitted	A Rover "sent" event — shows local submission only
Delivered to Earth	Earth server log and/or Earth packet capture
Round trip complete	A response matched at Rover by its Token
Aggregate	One captured bundle with two decoded CoAP members — a GUI display alone does not prove it

11 Wireshark: see the real BPv7 bundles

RUN ON EARTH

On Earth, capture the network traffic and inspect TCPCLv3, BPv7 three-component IPN EIDs and the embedded CoAP NON messages. The working reference environment used Wireshark 4.6.9; you need a build that can decode the relevant RFC 9758 BPv7 fields.

A Install the repository Lua dissector

Copy `wireshark/coapbp05_ipn64_PL65000.lua` into the **Personal Lua Plugins** directory shown in **Wireshark → Help → About Wireshark → Folders**. The location depends on whether Wireshark is native or Flatpak. Restart Wireshark to load the plugin.

B Choose how to capture

- **Native Wireshark:** select `enp0s8` directly if your installation exposes it and your user has capture permissions.
- **Flatpak Wireshark with no interfaces:** install Flatpak Wireshark plus host tcpdump (for example, `sudo apt install tcpdump`), then use the provided helper. It starts host capture and opens Wireshark; you do not type a separate tcpdump command.

EARTH CAPTURE TERMINAL · KEEP OPEN

```
cd ~/CoAPoverBP-IPN-Draft05-NoProxy
CAPTURE_FILTER="tcp port 4773" \
  bash wireshark/wireshark-ipn-live.sh enp0s8
```

The helper uses a named pipe and requires flatpak, tcpdump and sudo capture permission; it does not restart any network node.

C Display filters — use one at a time

Filter	Shows
`tcp.port == 4773`	All traffic to and from Earth's TCPCLv3 listener
`bpv7`	Decoded BPv7 bundles
`bpv7.eid.ipn_count == 3`	Bundles using three-component IPN EIDs
`bpv7.eid.ipn_alloc == 974848`	Bundles using this lab's allocator
`coapbp05.count == 1`	Bundles carrying one CoAP message (Single)
`coapbp05.count == 2`	Bundles carrying two CoAP messages (Aggregate)

Inspect the source and destination EIDs, allocator, node, service, MID, Token, and the optional experimental **Payload-length option 65000**. If TCPCL is shown only as TCP, use **Analyze → Decode As**. Save the packet trace as `.pcapng`.

> **Two decoders work together**
> The **coapbp05 Lua plugin** decodes the CoAP payload; Wireshark 4.6.x's **native BPv7 dissector** handles the three-component IPN EID fields. The two parts are complementary.

12 Challenge: simulate ON/OFF connectivity

RUN ON SATELLITE ROVER EARTH

Once the normal live exchange succeeds, test what happens when the Earth-facing contact becomes temporarily unavailable. This illustrates disruption-tolerant store-and-forward behaviour.



Two Science POSTs are generated while the contact is OFF — do they arrive when it returns?

Suggested participant experiment

Step	Action
1	Capture a normal Safety GET and Science POST with contacts **ON**. Record request and response times.
2	Using the existing GUI/contact controls where supported, and **only when authorized by the testbed owner**, set the Satellite–Earth contact **OFF** for a controlled period. Do not kill µD3TN or edit the IP settings.
3	Submit a new CoAP NON request during the OFF period. Observe whether the application submits the bundle, whether Earth receives it yet, and where evidence of buffering can be seen.
4	Restore the contact **ON**. Monitor Earth arrival and any response arriving back at Rover. Check timing, queueing delay and possible expiration.
5	Save the same-run Rover/Earth logs and Wireshark capture. Compare ON versus OFF results in the Results log.

Four runs to compare

Pattern	2 Singles	1 Aggregate
Always ON	Run 01 · baseline	Run 02 · baseline
ON → OFF → ON	Run 03 · outage	Run 04 · outage

What to capture

Question	Evidence
Was the request submitted?	Rover GUI AAP2 submission event, Token and MID
Did it reach Earth?	Earth server log plus BPv7 bundle capture
Did the reply return?	Matched response Token at Rover
Was it delayed or lost?	Timing evidence, contact state and bundle lifetime
Was it an Aggregate?	One BPv7 request bundle with two decoded CoAP members

Experimental caution

The provided default contact scripts configure hour-long contacts, not an automated ON/OFF schedule. Use existing GUI controls only where they are known to operate correctly in your instance. NSTART=2 is an experimental setting, not a general congestion-safety guarantee.

13 Troubleshooting: where to look first

Find the symptom, try the safe next check, and revisit the listed section. All checks below are read-only unless stated.

Problem	Likely cause / safe next check	See
Git clone shows the wrong code	Run `git branch --show-current`; it must be `IPN_RFC_9758_Hackathon`.	§02
No enp0s8 interface	Check VirtualBox Adapter 2 = intnet; run `ip -br link`; set `IETF127_NET_IFACE` if different.	§03
Ping fails	Check that all 3 VMs are on the same host/intnet, with unique /24 addresses.	§03–04
µD3TN patch/build fails	Save compiler output. Install build-essential and libsqlite3-dev; never force-patch or overwrite existing source.	§05
ImportError: protobuf runtime	Generated AAP2 file requires Protobuf 5.28.3. Correct the setup script pin and rerun setup.	§06
Tkinter missing	Install Tk support matching the selected Python; verify `python3.10 -c "import tkinter"`.	§02
Earth Python says ud3tn_utils missing	Run `source scripts/participant_env.sh` in that terminal and check `setup --live`; inspect PYTHONPATH.	§06
Rover GUI: AAP2 socket missing	Start patched Rover µD3TN first; check `/tmp/ud3tn-974848-rover.aap2.sock`.	§07
Request sent, no Earth response	Check daemon/contact logs, Earth server, time sync, relevant ports; inspect Earth Wireshark.	§08–11
Contact works then stops	Default schedule lasts 3,600 seconds; check whether it expired. Do not override active GUI-managed contacts.	§08
Flatpak Wireshark shows no interfaces	Use `wireshark/wireshark-ipn-live.sh` on Earth after installing host tcpdump and flatpak.	§11
Only TCP, not decoded BPv7	Check port 4773, TCPCLv3 Decode As, complete TCP stream, and plugin loading.	§11

ALL THREE VMs Read-only diagnostics to collect

```
ip -br -4 addr
ping -c 3 192.168.100.2
ss -ltn | grep 477
ps -eo pid,args | grep -E "[u]d3tn|[N]odeB_Mission|[m]ission_control"
date -u
```

When reporting a problem, include
The command you ran, the VM role, the last 20–30 lines of its output, and whether you were in LIVE or DEMO mode.

14 Completion checklist and source links

Tick each item as you go. Solid boxes are required for the live lab; dashed boxes are optional challenge outcomes.

- [ ] Downloaded `IPN_RFC_9758_Hackathon` on all three VMs
- [ ] Adapter 1 = NAT; Adapter 2 = Internal Network intnet on each VM
- [ ] Rover .3, Satellite .2 and Earth .1 internal IPv4 addresses verified
- [ ] All VMs can ping each other; clocks checked
- [ ] Patched µD3TN built successfully on each VM
- [ ] Protobuf 5.28.3 fix applied; Python LIVE imports pass
- [ ] Three µD3TN nodes running on TCPCLv3 ports 4771–4773
- [ ] Four contact directions configured via Satellite
- [ ] Earth NodeB server running and registered at `ipn:974848.3.64`
- [ ] Rover GUI opened with `bash ./run_gui_congestion_results.sh`
- [ ] Safety GET and Science POST received and responded to
- [ ] Wireshark captures saved, with allocator 974848 and matching EIDs
- [ ] Optional: one Aggregate bundle with two Science CoAP members verified
- [ ] Optional: ON/OFF contact results and corresponding logs saved

Authoritative project and protocol links

Hackathon repository branch
`https://github.com/ENTEL-WNG/CoAPoverBP-IPN-Draft05-NoProxy/tree/IPN_RFC_9758_Hackathon`

Pinned original no-proxy source
`https://github.com/ENTEL-WNG/CoAPoverBP-no-proxy-version`

Current installation documentation
`https://github.com/ENTEL-WNG/CoAPoverBP-IPN-Draft05-NoProxy/blob/IPN_RFC_9758_Hackathon/docs/installation.md`

Wireshark helper and Lua instructions
`https://github.com/ENTEL-WNG/CoAPoverBP-IPN-Draft05-NoProxy/blob/IPN_RFC_9758_Hackathon/wireshark/README.md`

CoAP over BPv7 — draft-ietf-core-coap-bp-05
`https://datatracker.ietf.org/doc/draft-ietf-core-coap-bp-05/`

RFC 9758 — three-component IPN addressing
`https://www.rfc-editor.org/rfc/rfc9758`

> **Research note**
> This is an experimental research prototype. The experimental CoAP Payload-length option 65000 and controlled NSTART=2 mode must not be presented as independently proven universal standards compliance. The recorded prepared-lab results are not a guarantee of identical fresh participant results.

MINISTERIO DE CIENCIA, INNOVACIÓN Y UNIVERSIDADES
Cofinanciado por la Unión Europea
AGENCIA ESTATAL DE INVESTIGACIÓN

This work was funded in part by the Spanish MCIU/AEI/10.13039/501100011033/FEDER/UE through project PID2023-146378NB-I00

AI tools assisted with the editing and layout of this participant manual.

A Glossary

The terms you will meet in the logs, the GUI and Wireshark.

BPv7	Bundle Protocol version 7 (RFC 9171). Moves data as self-contained bundles that nodes can store and forward when no end-to-end path exists.
Bundle	The BPv7 unit of transfer. In this lab a bundle carries one CoAP message (Single) or two (Aggregate).
Store-and-forward	A node keeps a bundle until a contact to the next hop is available, instead of dropping it.
Contact	A scheduled window during which a node may forward bundles to a neighbour (§08).
Bundle lifetime	How long a bundle stays valid after creation. Past it, the bundle may be discarded — relevant during long OFF periods.
µD3TN	The open-source DTN implementation that runs the BPv7 node on each VM, patched here for IPN/AAP2 (§05).
AAP2	µD3TN's Application Agent Protocol v2. Applications attach to their local µD3TN node through a Unix socket (`/tmp/ud3tn-974848-*.aap2.sock`).
TCPCLv3	TCP Convergence Layer v3. Carries bundles between neighbouring nodes over TCP (ports 4771–4773).
EID	Endpoint ID. Here a three-component IPN name, `ipn:allocator.node.service` (RFC 9758). Service 0 names the node itself; other services name applications.
Allocator 974848	The allocator identifier used by every EID in this lab. Filter on it in Wireshark with `bpv7.eid.ipn_alloc`.
CoAP NON	A non-confirmable CoAP message: sent without expecting an acknowledgement. This lab's verified baseline uses NON only.
MID	CoAP Message ID. The modified aiocoap used here supports a 24-bit MID.
Token	CoAP value that matches a response to its request. A matched Token at Rover proves the round trip.
NSTART	Maximum number of outstanding CoAP interactions toward a server. Baseline is 1; the Aggregate experiment uses 2 for evaluation only.
Single / Aggregate	Single: one CoAP request per bundle. Aggregate: two Science POSTs in one bundle, answered by two separate Single responses.
Payload-length option	Experimental CoAP option (number 65000) carried by each CoAP message in the prototype; visible in the coapbp05 dissector.
LIVE / DEMO	LIVE sends real traffic through the three nodes. DEMO shows simulated values for learning the GUI and is never evidence.
intnet	The VirtualBox Internal Network that links the three VMs on one host.

B Results log

Print this page or copy the table into your notes. Record UTC times from the same run's logs and captures (§12).

Run	Token(s) / MID(s)	Submitted at Rover (UTC)	Arrived at Earth (UTC)	Reply matched at Rover (UTC)	Bundles seen · files saved
Run 01 2 Singles · always ON					
Run 02 1 Aggregate · always ON					
Run 03 2 Singles · ON → OFF → ON					
Run 04 1 Aggregate · ON → OFF → ON					

OFF period (from – to, UTC)	Contact lifetime / bundle lifetime settings

OBSERVATIONS — SINGLES VS. AGGREGATE AFTER RECONNECTION

Mode: ☐ LIVE ☐ DEMO (not evidence) · Tester: _____ · Date: _____